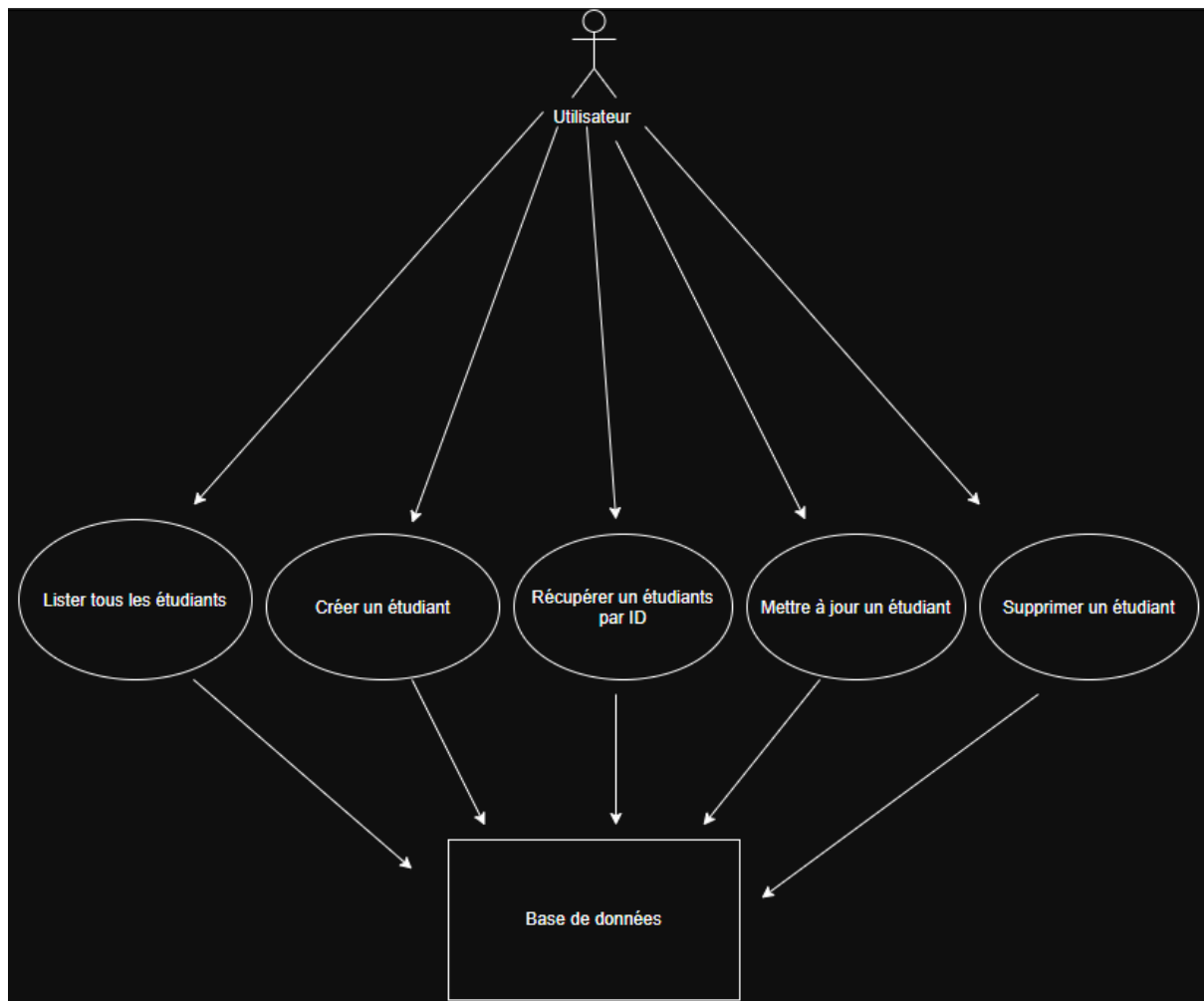
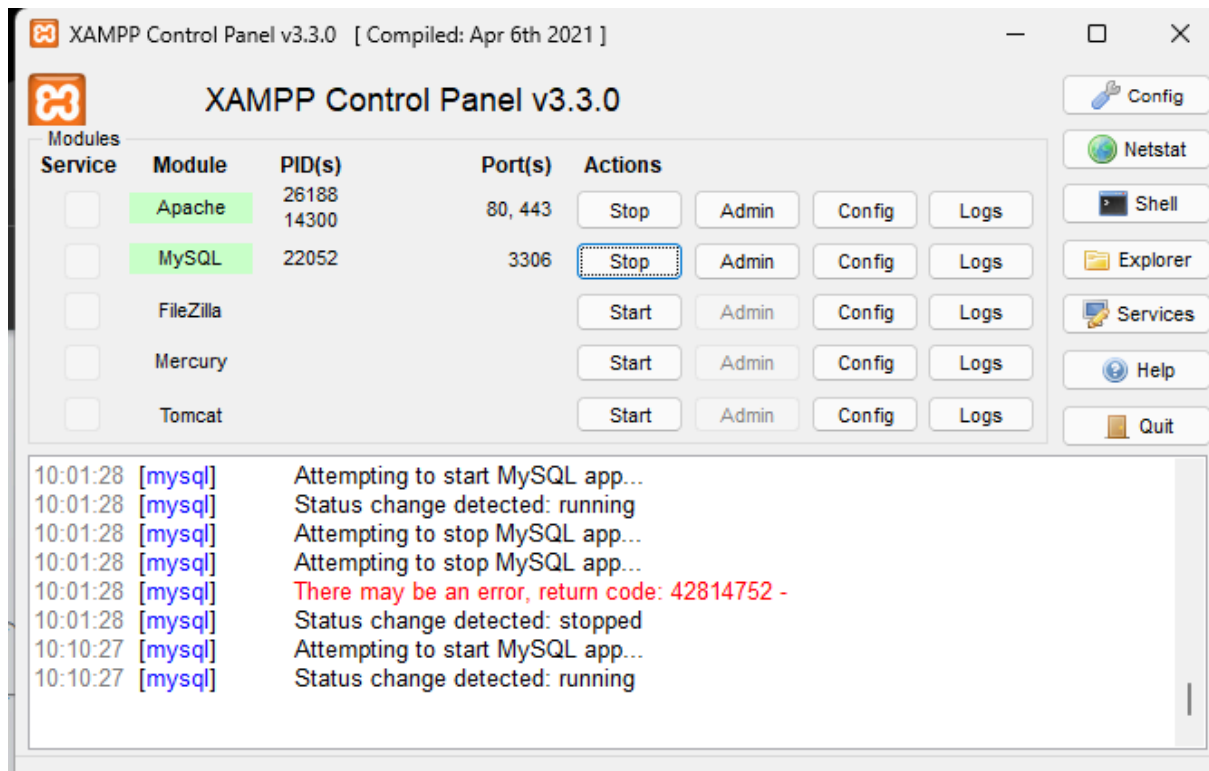


Rapport : Création d'une API REST

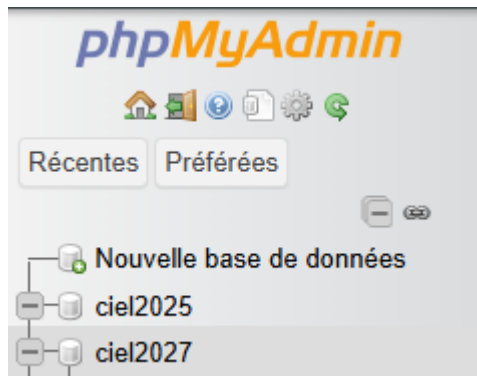
Objectif : Développer la partie “Back-end” d’une application en ligne permettant de gérer une liste d’étudiants.



Tout d’abord on doit lancer Apache et MySQL :



Ensuite on doit avoir sur phpmyadmin une base de donnée "ciel2027" :



Il faut qu'on est une table "étudiant"

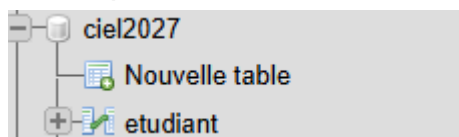


Table	Action	Lignes	Type	Interclassement	Taille	Perte
etudiant	Parcourir Structure Rechercher Insérer Vider Supprimer	11	InnoDB	utf8mb4_general_ci	16,0 kio	-

à l'intérieur :

			idetudiant	nom	prenom	email	telephone
☐		Éditer		Copier		Supprimer	
	1	ALEXANDRE	Thomas	thomas.alexandre97@gmail.com	+596696123456		
☐		Éditer		Copier		Supprimer	
	2	BOUTRIN	Mélissa	melissa.boutrin@gmail.com	+596696234567		
☐		Éditer		Copier		Supprimer	
	3	CHALONCYS	Stéphane	stephane.chaloncys@gmail.com	+596696345678		
☐		Éditer		Copier		Supprimer	
	4	DUMONT	Aurélie	aurelie.dumont972@gmail.com	+596696456789		
☐		Éditer		Copier		Supprimer	
	5	EUSTACHE	Yann	yann.eustache97@gmail.com	+596696567890		
☐		Éditer		Copier		Supprimer	
	18	TERIEUR	Alex				
☐		Éditer		Copier		Supprimer	
	19	TERIEUR	Alex				
☐		Éditer		Copier		Supprimer	
	22	TERIEUR	Alex				
☐		Éditer		Copier		Supprimer	
	23	TERIEUR	Alex				
☐		Éditer		Copier		Supprimer	
	25	TERIEUR	Alex				
☐		Éditer		Copier		Supprimer	
	26	Dupont	Jean				

Ensuite on doit coder une application en langage Python pour accéder à ses données

Api_v1:

https://github.com/florianedgard/API-rest-d-une-application/blob/main/api_v1.py

Requête pour voir tous les étudiants de la table

Méthode HTTP	Route	Fonction associée	Paramètres
GET	/v1/etudiants/	getEtudiants()	N/A

Réponses possibles

Code HTTP	Données / Message en format JSON
200	Tableau contenant tous les étudiants
400	Requête invalide
401	Accès non autorisé
500	Échec de connexion à la base de données

Requête pour voir un seul étudiant

Méthode HTTP	Route	Fonction associée	Paramètres
GET	/v1/etudiants/{id}	getEtudiant(id)	id de l'étudiant dans la route

Réponse

Code HTTP	Données / Message en format JSON
200	Détails de l'étudiant en format JSON
400	Requête invalide
401	Accès non autorisé
404	id invalide
500	Échec de connexion à la base de données

Requête pour ajouter un étudiant

Méthode HTTP	Route	Fonction associée	Paramètres
POST	/v1/etudiants/	addEtudiant()	Détails de l'étudiant en format JSON dans le body

Réponse

Code HTTP	Données / Message en format JSON
201	Ajout OK
401	Accès non autorisé
500	Échec de connexion à la base de données

Requête pour modifier les détails d'un étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
PUT	/v1/etudiants/{id}	updateEtudiant(id)	id de l'étudiant dans la route Détails de l'étudiant en format JSON dans le body
Réponse			
Code HTTP	Données / Message en format JSON		
200 401 404 500	Modification OK Accès non autorisé id invalide Echec de connexion à la base de données		

Requête pour supprimer un étudiant			
Méthode HTTP	Route	Fonction associée	Paramètres
DELETE	/v1/etudiants/{id}	deleteEtudiant(id)	id de l'étudiant dans la route
Réponse			
Code HTTP	Données / Message en format JSON		
200 401 404 500	Suppression OK Accès non autorisé id invalide Echec de connexion à la base de données		

On a les 4 méthodes Http pour les 5 cas d'utilisations

- GET:lister tous les étudiants
- POST:créer un étudiant
- GET: Récupérer un étudiants par ID
- PUT:Mettre à jour un étudiant
- DEL:Supprimer un étudiant

Api_v2 :

https://github.com/florianedgard/API-rest-d-une-application/blob/main/api_v2.py

On a rajouté les try et except pour mieux gérer les erreurs

Api_v3 :

https://github.com/florianedgard/API-rest-d-une-application/blob/main/api_v3.py

On a rajouté une fonction login et on a crée un utilisateur sur phpmyadmin user1 avec comme mot de passe 123456.

Options supplémentaires

iduser login password
1 user1 123456

Aussi on utilise Postman pour tester les requêtes :

ciel2027

- GET lister tous les étudiants
- POST créer un étudiant
- GET Récupérer un étudiants par ID
- PUT Mettre à jour un étudiant
- DEL Supprimer un étudiant

POST {{base_url}} / {{api_version}} /etudiants

Docs Params Authorization Headers 9 Body Scripts Settings

Auth type Basic Auth

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

Username user1

Password 123456

Body Cookies Headers 5 Test Results 1/1 201 Created 7 ms 20

```
{
  "message": "Ajout OK"
}
```

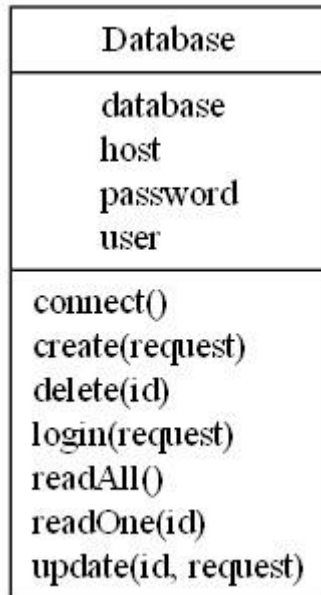
Api v4:

https://github.com/florianedgard/API-rest-d-une-application/blob/main/api_v4.py

J'ai transférer les fonctions de requête dans une classe dans une fichier db :

<https://github.com/florianedgard/API-rest-d-une-application/blob/main/db.py>

Un diagramme de classes pour la classe Database généré par pyreverse :



une fiche de test de l'API sur postman de la v3 :

GET lister tous les étudiants http://127.0.0.1:5000/v3/etudiants	200 • 7 ms • 1.178 KB • 2
PASS La requête a été coorectement exécutée	
PASS La requête est bien un tableau JSON	
POST créer un étudiant http://127.0.0.1:5000/v3/etudiants	201 • 10 ms • 198 B • 1
PASS L'étudiant est créé (Statut 201)	
GET Récupérer un étudiants par ID http://127.0.0.1:5000/v3/etudiants/20	200 • 5 ms • 262 B • 2
PASS La requête a été coorectement exécutée	
PASS La requête est bien un tableau JSON	
PUT Mettre à jour un étudiant http://127.0.0.1:5000/v3/etudiants/20	200 • 8 ms • 204 B • 1
PASS L'étudiant est créé (Statut 200)	
DELETE Supprimer un étudiant http://127.0.0.1:5000/v3/etudiants/20	200 • 8 ms • 199 B • 1
PASS L'étudiant est créé (Statut 200)	

une fiche de test de l'API sur postman de la v4 :

Iteration 1				
GET	lister tous les étudiants			
http://127.0.0.1:5000/v4/etudiants		200	191 ms	1.736 KB 2
PASS	La requête a été coorectement exécutée			
PASS	La requête est bien un tableau JSON			
POST	créer un étudiant			
http://127.0.0.1:5000/v4/etudiants		201	14 ms	210 B 1
PASS	L'étudiant est créé (Statut 201)			
GET	Récupérer un étudiants par ID			
http://127.0.0.1:5000/v4/etudiants/23		200	13 ms	262 B 2
PASS	La requête a été coorectement exécutée			
PASS	La requête est bien un tableau JSON			
PUT	Mettre à jour un étudiant			
http://127.0.0.1:5000/v4/etudiants/23		200	10 ms	209 B 1
PASS	L'étudiant est créé (Statut 200)			
DELETE	Supprimer un étudiant			
http://127.0.0.1:5000/v4/etudiants/23		200	15 ms	207 B 1
PASS	L'étudiant est créé (Statut 200)			